

The Future of Mathematics:

From Problem-Solving to Theory-Development in the Age of AI

Ashwin Rao

Adjunct Professor, ICME, Stanford University | Chief Technology Officer, o9 Solutions

September 2026

ABSTRACT

Mathematics has spent centuries getting progressively better at solving precisely stated problems, while remaining remarkably informal about how new mathematical worlds are created. This essay argues that modern AI has crossed a historic threshold in the former activity without thereby mastering the latter. Problem-solving searches for a proof, algorithm, model, program, or construction inside a sufficiently specified space. Theory-development chooses the objects, representations, definitions, questions, and standards of value that make such a space possible. We trace this distinction through a career spanning olympiad mathematics, algorithms and abstract algebra, derivatives trading, retail optimization, programming, reinforcement learning, teaching, and exposition. The result is a specification frontier: AI is already formidable when the mathematical "what" is supplied; it remains far less autonomous in deciding what the mathematical "what" should be.

The central claim: AI has crossed a threshold in well-specified mathematical problem-solving. It has not thereby crossed the threshold into autonomous theory-development.

1 | The future of mathematics

We are on the verge of a more fundamental shift in mathematics than most discussion currently appreciates. Not just better calculators, faster proof search, or more fluent tutoring - but a change in how mathematical labor is divided.

Claims that AI is 'taking over mathematics' nevertheless move too quickly from a spectacular result to an expansive conclusion. A system solves an olympiad problem, fills a Lean proof, or improves a combinatorial bound; the accomplishment is then treated as evidence that the occupation of mathematician is nearing automation. But an answer to a mathematical question is not the same object as a mathematical theory, and proving a theorem is not the whole of mathematical progress.

William Thurston made this point long before modern AI. He resisted a picture of mathematics as a pipeline of definitions, theorems, and proofs, arguing instead that mathematicians advance human understanding. The source of the questions, the choice of language, and the development of usable ways of thinking are not peripheral to the enterprise; they are much of the enterprise (Thurston, 1994). Today's debate risks reinstating the narrow picture precisely because what machines can now do most visibly is produce answers and proofs.

The right response is not to minimize the new systems. Their results are too strong for that. Nor is it to protect a mystical category of 'creativity' that is human by definition. The useful response is to decompose mathematical work more accurately. My proposed decomposition begins with two modes: problem-solving and theory-development. They overlap, and the boundary between them is not perfectly sharp. Nevertheless, the distinction explains both why current achievements are historic and why they leave a substantial part of mathematics untouched.

2 | Two modes, one confusion

Problem-solving

In problem-solving, the mathematical world has largely been chosen in advance. The objects are named; the admissible operations are understood; a question is posed; and there is a reasonably crisp notion of success. The problem may be brutally difficult. It may require a construction nobody has seen, an ingenious invariant, or a proof that runs for hundreds of pages. Difficulty is not the distinguishing variable. Specification is.

A compact formalization is useful. Let L denote a language, A a set of accepted assumptions and methods, G a goal, and V a verifier or evaluative criterion. Problem-solving asks for an artifact p - a proof, algorithm, model, program, or construction - such that $V(p; L, A, G)$ accepts it or scores it highly. Olympiad mathematics is a clean example. So are much of algorithm design, quantitative modeling, and programming once the requirements have stabilized. The search space may be vast, but the game is identifiable.

Theory-development

Theory-development acts one level above this search. It chooses or revises L , A , G , and sometimes V . It asks what the right objects are, what should count as the same object, which features are essential, what a productive definition should preserve, which questions deserve attention, and what kind of result would explain rather than merely certify. A successful theory does not simply settle one goal. It creates a landscape in which many goals become visible, related, and tractable.

This work includes problem-posing, representation design, definition invention, analogy, abstraction, conjecture formation, the organization of lemmas into a reusable architecture, and judgments of mathematical taste. Its outputs are often evaluated only retrospectively. A definition that initially looks artificial may become indispensable because it supports decades of theorems. A theorem may be correct yet sterile. A technically weaker result may matter more because it reveals the right coordinate system. Theory-development is therefore not just harder search. It partly constructs the search space and the value function.

Dimension	Problem-solving	Theory-development
Objective	Supplied or tightly bounded	Chosen, reframed, or created
Vocabulary	Mostly fixed	Invented, transferred, or reorganized
Evaluation	Correctness or score is available	Interest and fertility are partly endogenous
Typical output	Answer, proof, algorithm, model	Framework, abstraction, research program
Failure mode	Incorrect or suboptimal result	Correct but unilluminating or sterile theory
Time horizon	Usually bounded	Open-ended and socially validated

Table 1. The distinction is about who specifies the mathematical world, not about how difficult the resulting theorem is.

A reinforcement-learning analogy

For someone trained in reinforcement learning, the distinction can be stated provocatively. Problem-solving resembles optimizing a policy inside a given Markov decision process. The states, actions, dynamics, and reward may be complex or only partially known, but they define the task. Theory-development resembles deciding what the state should be, which distinctions can be discarded, what actions are meaningful, where the boundary of the environment lies, and what reward captures the purpose. In real work, model and policy are developed together. But a breakthrough in policy optimization does not by itself show mastery of model or objective design.

This analogy also explains why the distinction is not metaphysical. Humans do not possess a magic faculty called theory. We perform a collection of meta-level activities for which present AI systems have weak or indirect training signals. Those signals can improve. The claim of this paper is empirical and architectural, not a proof of permanent human exceptionalism.

3 | Evidence from one mathematical life

My claim to perspective is not greatness but range. I have encountered mathematics as a high-school olympiad contestant, a doctoral student in computer science working in algorithms, discrete mathematics, and abstract algebra, a quantitative practitioner in finance, a mathematical modeler in retail, a programmer and software designer, an author, and an adjunct professor teaching reinforcement learning and mathematical finance. Across those roles, the two modes felt recognizably different (Rao and Jelvis, 2022; Rao, 2026a).

The distinction is not between pure and applied mathematics, nor between academia and industry. Both modes occur in every setting, and often in the same piece of work. The distinction is between solving inside a mathematical world and constructing or reorganizing that world.

The posed problem: olympiads, models, algorithms, and notebooks

Olympiad work is the purest memory of problem-solving. The problem arrives as a small, finished universe. All attention goes to the route from statement to solution. The same feeling recurs in professional mathematics after a business objective has been translated into a model: price this security under these assumptions; design an execution policy with these costs; allocate inventory subject to these constraints; implement an algorithm that meets these performance requirements. There is creativity in every case, and sometimes formidable creativity. Yet the destination is externally legible.

Many of the Colab notebooks on my Stanford page occupy this side of the ledger. They formulate and compute answers for retirement planning, inventory control, clearance pricing, price elasticity, optimal order placement, cricket batting strategy, and related decisions. They are useful precisely because they close the loop from a stated model to a reproducible answer. The coding can be delicate and the modeling consequential. But once the variables, assumptions, and objective are settled, the remaining work is chiefly to solve the specified problem (Rao, 2026a).

AI is increasingly at home in this regime. It can explore cases, generate and test code, search proof branches, propose auxiliary constructions, compare algorithms, and formalize the result. In programming, it can often traverse the path from specification to working artifact. For many routine and even elite posed problems, the human comparative advantage has already narrowed sharply.

The Newsvendor meets the Options Trader

A different kind of mathematical act appears in my note The Newsvendor meets the Options Trader. A derivatives trader describes a call payoff as the expectation of $(x - S)^+$ and a put payoff as the expectation of $(S - x)^+$. A retail optimizer describes overage and underage costs around an order quantity S . Surface vocabulary says these are different domains. The equations say they are the same object: a portfolio of weighted calls and puts, minimized over a common strike, is the newsvendor optimization problem (Rao, n.d.).

Once that equivalence is seen, ideas move across the bridge. Put-call parity becomes a statement about underage and overage costs. Moneyness becomes the distance between expected demand and supply. Option delta becomes sensitivity of inventory cost to expected demand. At the optimum, the familiar cycle-stock and safety-stock decomposition has a market-neutral analogue. None of these calculations is individually mysterious. The theory-developing move is choosing a representation in which the parallel becomes inevitable rather than surprising.

This is the sort of mathematics I found most valuable when moving from fourteen years in derivatives trading to supply-chain optimization. One does not merely carry formulas from Finance into Retail. One finds an abstraction under which both become special cases. Thereafter, many downstream tasks return to problem-solving.

Recasting financial decisions as Markov decision processes

Problems I had worked on at Goldman Sachs and Morgan Stanley were not originally presented through the lens of stochastic control. Years later, while designing Stanford's CME 241 and the book Foundations of Reinforcement Learning with Applications in Finance, I recognized that several could be cast as Markov decision processes. That recasting connected portfolio decisions, derivatives hedging, optimal execution, market making, and even retail inventory and clearance pricing to a common language of state, action, reward, value, and Bellman operators (Rao and Jelvis, 2022).

Once the recasting is made, much becomes problem-solving: derive the Bellman equation, choose an approximation architecture, prove convergence under assumptions, write the code, tune the policy. But identifying the common frame changes which questions can be asked. It imports results from one domain into another, exposes shared structure, and makes apparently different applications instances of the same object. That is applied theory-development. It need not create a new branch of pure mathematics to exhibit the same kind of thought.

Function approximations as affine spaces

The appendix Function Approximations as Affine Spaces makes a similar move at a more abstract level. A function approximation is usually presented as a parameterized formula to be fitted. The appendix instead treats the parameter domain as an affine space, the gradients as its associated vector space of translations, and the family of function approximations as a corresponding delegator space. Stochastic gradient descent can then be understood as translating a point in that affine space (Rao and Jelvis, 2022, Appendix F).

The practical algorithm has not changed. The representation has. The gain is conceptual economy: linear approximators and neural networks can share one language despite having different parameter containers, and a software operation such as adding a gradient acquires a precise mathematical meaning. This is theory-development in miniature - not because the abstraction is grand, but because it turns a family of computations into instances of a structure.

Category theory and abstract algebra: teaching the habit of theory

My notes on Abstract Algebra and Category Theory are expository rather than claims of original theory. They nevertheless reveal what theories are for. Abstract algebra asks us to forget what elements are made of and retain the operations and laws they preserve. Category theory moves again, from objects to morphisms, composition, functors, and natural transformations. The reward is not one solved exercise. It is the ability to recognize that apparently unrelated constructions are structurally the same (Rao, 2016; Rao, 2017).

Teaching these subjects also exposes why theory-development is hard to evaluate locally. A definition can be formally correct and pedagogically dead. The learner understands only when canonical examples, non-examples, diagrams, and compositional relationships make the abstraction usable. Theory must create a way of seeing, not merely a vocabulary.

The Future of Programming goes one level up

My recent paper The Future of Programming argues that AI will increasingly compile a human statement of what a program should do into a verified implementation of how it does it. The central object becomes a formal problem specification rather than implementation code (Rao, 2026b). The present paper applies the same architecture to mathematics, then moves one level higher.

A formal theorem statement is a problem specification; a proof is its implementation. AI is becoming an extraordinarily capable compiler from the former to the latter. But theory-development creates and revises the specifications themselves: the language in which the theorem can be stated, the definitions that make it natural, the target that makes it worth proving, and the standards by which the result will matter. AI may take over the mathematics of the how long before it takes over the mathematics of the what.

Abstraction as infrastructure

The reinforcement-learning book offered another version of the distinction. Its codebase was designed so that software abstractions map to mathematical concepts and relationships. A well-designed interface is not merely a programming convenience; it is a claim about what is invariant across examples and what should be hidden. The work is successful if new algorithms and applications fit naturally without destroying clarity. This is close to what a good mathematical definition does. Both are forms of intellectual infrastructure.

The lesson from moving among abstract algebra, finance, retail, and software is that the most valuable move is often a change of representation. A finance practitioner and a retail operator may use different nouns while facing homologous decisions under uncertainty. The bridge appears only when someone ignores the surface vocabulary and preserves the right structure. AI can already suggest analogies. It is much less reliable at deciding which analogy deserves to reorganize the work of a field or an enterprise.

4 | What the strongest AI results actually show

The distinction must survive the strongest evidence against it. Weak examples prove little. As of September 2026, four lines of work are especially important.

Olympiad mastery is now real

In 2024, Google DeepMind's AlphaProof and AlphaGeometry 2 solved four of six International Mathematical Olympiad problems for a score equivalent to a silver medal. The problems were manually translated into formal languages, and some solutions required days of computation. One year later, an advanced Gemini system produced natural-language proofs for five of six IMO problems within the competition time limit and received an officially certified gold-medal score (Google DeepMind, 2024; Google DeepMind, 2025b). Aristotle separately combined informal reasoning, lemma generation, formalization, and Lean proof search to report gold-equivalent performance on the 2025 problems (Achim et al., 2025).

It would be evasive to describe these systems as mere pattern matchers. They generate non-obvious constructions, decompose goals, create auxiliary lemmas, and produce extended arguments. Whatever human cognition ultimately is, the practical conclusion is clear: olympiad problem-solving is no longer a safe benchmark for uniquely human mathematical intelligence.

Research-level proof search has crossed a second threshold

The more serious challenge to my thesis arrived in 2026. AlphaProof Nexus agents were evaluated on hundreds of open problems. The strongest system autonomously resolved 9 of 353 attempted Erdos problems, proved 44 of 492 open OEIS conjectures, and contributed to results in algebraic geometry and optimization (Tsoukalas et al., 2026). Some questions had been open for decades. This is not competition mathematics; it is frontier mathematics.

Yet the architecture matters. The agent receives a Lean file containing the target theorem, definitions, imports, and a proof hole. Human-provided markers delimit what may change. Lean supplies an exact verifier. The agent can introduce helper lemmas and definitions, which is an important expansion of autonomy, but the overarching goal and formal world are given. The result demonstrates frontier problem-solving, not autonomous field construction. This is not a dismissal. It is a more precise description of an extraordinary advance.

Open-problem discovery is possible when evaluation is cheap

FunSearch and AlphaEvolve move further toward discovery. FunSearch improved known constructions for the cap-set problem and found better online bin-packing heuristics. Crucially, its authors characterize the target class as problems that are hard to solve but easy to evaluate; the system begins with an evaluator and usually a program skeleton (Romera-Paredes et al., 2024). AlphaEvolve likewise couples language models to automated evaluators and has improved matrix-multiplication algorithms, open mathematical bounds, and production computing systems. Its stated sweet spot is work whose progress can be measured objectively (Google DeepMind, 2025a).

These systems produce genuine novelty, sometimes beyond the best human result. But novelty is not identical to theory-development. They search for better inhabitants of a human-specified space under a human-specified score. They can discover a remarkable algorithm without deciding why that family of algorithms should matter, which conceptual vocabulary best explains the result, or which neighboring questions form a durable research program.

AI is beginning to enter the theory loop

The boundary is porous. In work on knot theory and representation theory, machine learning helped identify previously unseen relationships and salient features. Human mathematicians used those signals to introduce a new quantity, formulate conjectures, and prove results. The authors explicitly present the process as iterative collaboration: people choose candidate relationships and judge which patterns are interesting, while models expose structure in data too large for unaided inspection (Davies et al., 2021).

This is the most plausible near-term future: not a clean partition in which humans theorize and machines calculate, but a coupled loop. AI proposes patterns, counterexamples, lemmas, formal proofs, and candidate definitions. Humans revise the ontology and the objective. Over time, more of that revision may also be delegated. The honest claim is therefore not that AI contributes nothing to theory, but that current demonstrations remain scaffolded by human problem selection, representation, and value judgment.

5 | Why theory-development remains a different frontier

It chooses what to ignore

Abstraction is disciplined forgetting. A state representation excludes most facts about the world; a group forgets the material nature of its elements; a financial model discards microstructure it deems irrelevant; a retail model compresses millions of operational details into variables on which a decision can act. The difficult judgment is not whether information can be retained, but whether it should be. Good theory removes detail without destroying the phenomena we care about.

It invents value before value can be measured

Current AI systems advance fastest where candidate outputs can be scored automatically: a proof type-checks, a construction has a measurable size, a program runs faster, or a policy earns more reward. A theory can be correct in every local statement and still be useless. Its value includes explanatory compression, extensibility, transfer, pedagogy, and the questions it makes natural. These properties are real, but they are delayed, contextual, and partly social. The evaluator is one of the things theory-development must create.

It changes the unit of progress

Problem-solving counts solved goals. Theory-development reorganizes the relationship among goals. A new invariant can make a collection of theorems instances of one phenomenon. A definition can convert ad hoc tricks into a method. A categorical reformulation can reveal that two fields were manipulating

the same structure under different names. The value of the move may lie less in the first theorem proved with it than in the hundredth theorem someone else can now imagine.

It is reflexive and communal

Mathematics is not only a formal corpus. It is a living system of attention, explanation, notation, examples, standards, and trust. Researchers learn which gaps are routine, which questions are fashionable but shallow, and which strange definition may be worth years. Theory-development changes that social system and is evaluated by it. An AI could eventually participate in this loop, but benchmark success on isolated tasks does not yet measure such participation.

It requires sustained identity across failure

A research program often survives long periods without a clean reward. Definitions are revised; examples that looked central become distractions; a failed conjecture reveals the right obstruction; a proof technique is abandoned but its viewpoint endures. The agent must remember not only what happened but why the program was worth pursuing. Long context is helpful, but persistence of purpose is not the same as storage. It requires a model of what counts as progress while that model itself is changing.

6 | Objections that the thesis must absorb

'Inventing lemmas and definitions is already theory-development.'

Sometimes it is. A distinction by labels alone would fail. The relevant issue is scope. Introducing a local definition to close a supplied proof goal is a theory-like move inside problem-solving. Inventing a concept that changes the natural questions, transfers across settings, and supports an unanticipated body of results is a stronger form. Current systems show the first and occasional glimpses of the second. The continuum should be acknowledged rather than collapsed.

'Solving open problems disproves the distinction.'

An open problem can be more difficult than creating a theory, but it remains a posed problem. Erdos supplied extraordinarily fertile goals; formalizers supplied precise statements and libraries; proof systems supplied verification. Solving such a problem is research-level mathematics and deserves credit. It does not answer who chose the problem, why it mattered, or whether the solution yields a new explanatory framework.

'Humans also learn taste from data and communities.'

Yes. Nothing here requires a non-computational soul. Human mathematical taste is trained by examples, mentors, incentives, and feedback. In principle, machines may learn similar judgments from the literature and from interaction with mathematicians. The present difference is that our best automated feedback is dense for correctness and sparse for long-run fertility. That is an engineering and scientific gap, not necessarily an eternal barrier.

'A sufficiently capable agent can optimize the specification itself.'

This is the deepest objection and may ultimately be right. One can imagine agents that propose definitions, simulate downstream theorem yield, solicit expert feedback, revise their objectives, and maintain research programs for years. But that proposal concedes the present argument: such an agent needs capabilities and evaluation environments quite different from those demonstrated by solving fixed benchmarks. We should build and test them directly rather than infer them from medal counts.

7 | From a binary to a specification frontier

The two-part classification is most useful when refined into a frontier. Mathematical tasks vary along at least three axes: who selects the goal, who supplies the representation, and who supplies the evaluator. A closed task fixes all three. A more autonomous task allows the system to revise one or more. Theory-development lies toward the open end, where goals, representations, and standards co-evolve.

Level	Goal	Representation	Evaluation
1	Fixed exercise	Fixed	Exact verifier
2	Fixed theorem	Mostly fixed; lemmas open	Formal or expert check
3	Open problem	Fixed domain; methods open	Target result specified
4	Research direction	Definitions may change	Mixed and delayed
5	Field construction	Ontology is invented	Fertility judged over time

Table 2. A proposed specification frontier. Current systems are strong at Levels 1-2, increasingly consequential at Level 3, and experimentally entering Level 4.

This framing avoids two errors. It does not reserve 'real mathematics' for humans whenever AI succeeds. Nor does it confuse movement on one axis with completion of the whole journey. AlphaProof

Nexus at an open Erdos problem is Level 3 even if the proof is profound. An AI-human system that discovers a useful invariant and reorganizes a research agenda may be entering Level 4 even if its first theorem is elementary.

8 | What would count as AI theory-development?

If the thesis is scientific rather than protective, it needs falsification criteria. I would substantially revise my position if an AI system repeatedly demonstrated the following behaviors with limited human scaffolding.

Independent problem choice. From a broad and messy literature, the system identifies neglected questions that experts later judge important, rather than selecting from a prepared benchmark.

Definition invention. It introduces concepts not merely as proof conveniences but because they compress examples, expose invariants, and support unexpected theorems.

Cross-domain transfer. It recognizes a structural equivalence between fields whose surface vocabularies and datasets differ, then uses the bridge to generate new results on both sides.

Research-program persistence. It sustains and revises a line of inquiry over months, learns from failed conjectures, and maintains a coherent account of why the program matters.

Explanatory communication. It produces representations and expositions that change how expert humans think, not only proofs that they can verify.

Downstream fertility. Independent researchers adopt its concepts and obtain results the originating system did not target or foresee.

These criteria are intentionally harder to benchmark than theorem accuracy. That difficulty is part of the point. We have optimized AI for environments with cheap feedback because those environments support scaling. Theory-development asks us to build systems that operate when feedback is delayed, contested, and partly produced by the work itself. A benchmark may need longitudinal expert panels, blind comparison of alternative conceptual frameworks, and measurement of downstream reuse rather than one-shot correctness.

9 | A realistic human-AI research pipeline

Even if AI remains weak at autonomous theory-development, it will transform how theories are built. A mathematician with a proof-search engine, counterexample generator, formalizer, literature navigator, symbolic calculator, and tireless coding partner works differently from one without them. The machine can make conjecture-refutation cycles faster, reveal patterns in large spaces, and remove much of the

clerical burden of rigor. Human attention can move toward framing, abstraction, interpretation, and communication.

That division will not stay fixed. AI will absorb progressively more local theory-like acts. Humans, in turn, will learn to specify less and collaborate at higher levels. The relevant unit may become the human-AI research system rather than either participant. The 2021 knot-theory and representation-theory work already points in this direction: machine pattern recognition and human conceptual judgment form a loop, with neither contribution reducible to the other.

For education, the implication is uncomfortable but constructive. Training students primarily to outperform machines on closed-form exercises is no longer defensible. Students still need technique; without it, they cannot recognize a bad abstraction or inspect an AI proof. But curricula should put more weight on problem formulation, examples and counterexamples, definition design, model criticism, cross-domain analogy, exposition, and the reconstruction of why a theorem matters. The goal is not to preserve human jobs by retreating to vagueness. It is to teach the part of mathematics that our exams have historically under-measured.

For industry, the same principle applies. Once an optimization problem is faithfully specified, AI can increasingly build and solve the machinery. The largest failures often occur earlier: optimizing the wrong objective, omitting a critical state variable, treating an organizational constraint as a law of nature, or confusing a measurable proxy with enterprise value. In finance, retail, and supply-chain planning, the mathematics of choosing the model can dominate the mathematics of solving it. Neuro-symbolic systems and enterprise knowledge models are attempts to make that specification explicit; their success will depend as much on conceptual architecture as on inference power (o9 Solutions, 2026).

10 | Conclusion

AI has not merely become better at school mathematics. It has reached olympiad gold, proved open conjectures, and generated novel algorithms and constructions. Any serious position must begin by granting the magnitude of that achievement. In a growing class of well-specified mathematical tasks, 'taking over' is not an absurd description of the direction of travel.

But mathematics is not the set of well-specified tasks. Its deepest progress often consists of changing the specification: inventing an object, finding a representation, selecting a question, importing a structure across domains, or giving a community a new way to think. Current AI systems participate in fragments of this process, usually inside human-built frames with human-supplied evaluators. They are not yet autonomous authors of mathematical worlds.

I would not bet on permanent human monopoly. The history of AI warns against defining intelligence as whatever machines have not yet done. I would, however, insist on naming the missing capability accurately. The next decisive milestone is not another medal, a longer proof, or even another solved open problem. It is a system that can decide what is worth formalizing, invent the language that makes it visible, and sustain a fruitful theory whose value was not already encoded in its reward.

The through-line from programming to mathematics is clean. In programming, the human states the proposition and AI increasingly finds the implementation. In mathematics, the human states the theorem and AI increasingly finds the proof. Theory-development begins one level earlier: with the invention of the proposition, the language, and the reason to care.

The future of mathematics is not less human because machines solve more problems. It is more rigorously focused on who sets the problem - and who creates the world in which the problem can be seen.

References

- Achim, T., et al. (2025). Aristotle: IMO-level automated theorem proving. arXiv:2510.01346. [arXiv](#)
- Davies, A., Velickovic, P., et al. (2021). Advancing mathematics by guiding human intuition with AI. Nature, 600, 70-74. [Nature](#)
- Google DeepMind. (2024). AI achieves silver-medal standard solving International Mathematical Olympiad problems. [Google DeepMind](#)
- Google DeepMind. (2025a). AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms. [Google DeepMind](#)
- Google DeepMind. (2025b). Advanced version of Gemini with Deep Think officially achieves gold-medal standard at the International Mathematical Olympiad. [Google DeepMind](#)
- o9 Solutions. (2026). o9 Solutions promotes Dr. Ashwin Rao to Chief Technology Officer. [o9 Solutions](#)
- Rao, A. (n.d.). The Newsvendor meets the Options Trader. [Source and PDF](#)
- Rao, A. (2016). Abstract Algebra in 3 Hours. [Slide deck](#)
- Rao, A. (2017). Category Theory Made Easy with (Ugly) Pictures. [Slide deck](#)
- Rao, A. (2026a). Papers, slides, articles, and Colab notebooks. [Stanford work archive](#)
- Rao, A. (2026b). The Future of Programming: From Problem Specifications to AI-Compiled Implementations - A Type-Theoretic Foundation. [Source and PDF](#)
- Rao, A., and Jelvis, T. (2022). Foundations of Reinforcement Learning with Applications in Finance. CRC Press. [Open-access author PDF](#)
- Romera-Paredes, B., et al. (2024). Mathematical discoveries from program search with large language models. Nature, 625, 468-475. [Nature](#)
- Thurston, W. P. (1994). On proof and progress in mathematics. Bulletin of the American Mathematical Society, 30(2), 161-177. [arXiv](#)
- Tsoukalas, G., Kovsharov, A., Shirobokov, S., et al. (2026). Advancing mathematics research with AI-driven formal proof search. arXiv:2605.22763. [arXiv](#)

Author's note. *This is a position paper: its central taxonomy and forecasts are arguments, not settled empirical findings. AI capabilities described here are current through 7 September 2026.*